



wBMS-Toolkit-API Guide

26/01/26

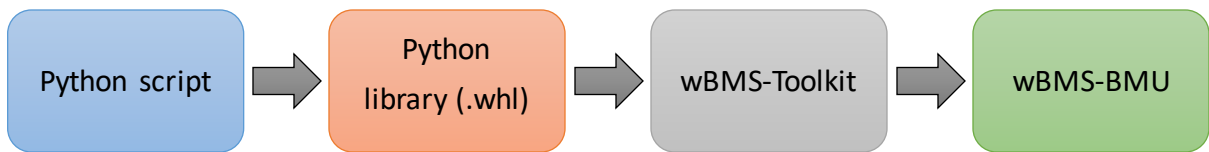
Confidential

The contents of this document are strictly confidential and cannot be distributed without the express written approval from Wattius.

1. Introduction

The main goal of this guide is to explain how to install and use the *wBMS-Toolkit-API*.

The *wBMS-Toolkit-API* is a Python package that allows to interact with the *wBMS-Toolkit* in real-time. Using the API, the client can program automatic processes to speed up some end of line processes during the production.



In the current version of the *wBMS-Toolkit-API*(v1.0.0), the next functionalities are implemented:

- Upload BMU Settings
- Apply BMU Setup
- Update Firmware
- Change connection mode.
- Start/Stop logging.
- Force OCV SOC
- Sign In
- Get Configuration CRC
- Get assigned IP
- Request wBMS-Toolkit-API status

2. Installation

The WBMS-Toolkit-API is compatible with Python 3.

Before starting the installation, check to upgrade the “pip” to the last version. To upgrade the “pip”, open the “CMD” and execute the next line:

```
pip install -upgrade pip
```

To start the installation, the WBMS-Toolkit-API library shall be downloaded from the Wattius Services repository. The WBMS-Toolkit-API file name follows the format *wbms_api-1.0.0-py3-none-any.whl* and must not be modified.

Once the API is downloaded, open the “CMD” and execute the next line:

```
pip install path/to/file.whl
```

Finally, create/open a python script to use the API and write the next lines:

```
from wbms_api.wbms_toolkit_api import *  
from wbms_api.wbms_toolkit_types import *
```

All the functions of the API can be used once these steps are completed.

3. Functions

The defined functions become available after importing, as explained in section 2, the "wbms_api.wbms_toolkit_api".

3.1. wbms_upload_configuration

- Parameters
 - config_file_path: String containing the path to the configuration file (.wconfig) that shall be uploaded.
- Return
 - result: Result of the upload configuration process. The result is returned as "wbms_result" enumeration.
 - OK: It means that the wBMS-Toolkit has uploaded correctly the configuration to the wBMS-BMU.
 - ERROR: It means that the wBMS-Toolkit has not uploaded correctly the configuration to the wBMS-BMU. In this situation, the cause of the problem can be any of the following points:
 - Incorrect configuration file.
 - The wBMS-BMU is not connected to wBMS-Toolkit.
 - The connection settings from the wBMS-Toolkit are not the same as the physical connection with the BMU.
 - TIMEOUT: It means that the uploading process has taken longer than expected. A typical situation may be when using an incorrect configuration file.
 - GETTING: It means that the wBMS-Toolkit is getting the current configured parameters from the wBMS-BMU. For this reason, the new requested configuration cannot be uploaded.
 - WAITING: It means that the wBMS-Toolkit has not any BMU connected or is not authorized to establish communication with it.

3.2.wbms_update_firmware

- Parameters
 - firmware_file_path: String containing the path to the firmware file (.srec) that shall be updated.
- Return
 - result: Result of the update firmware process. The result is returned as "wbms_result" enumeration.
 - OK: It means that the WBMS-Toolkit has updated correctly the firmware to the WBMS-BMU.
 - ERROR: It means that the WBMS-Toolkit has not been able to correctly update correctly the firmware to the WBMS-BMU. In this situation, the cause of the problem can be any of the following points:
 - Incorrect firmware file.
 - The WBMS-BMU is not connected to WBMS-Toolkit.
 - The connection settings from the WBMS-Toolkit are not the same as the physical connection with the BMU

3.3.wbms_apply_setup

- Parameters
 - canbus_baudrate: "wbms_can_baudrate" type indicating the desired can bus baud rate for the connected BMU.
 - canbus_id: Integer type indicating the desired can node ID for the connected BMU.
 - bluetooth_mode: "wbms_ble_mode" type indicating the desired bluetooth mode for the connected BMU.
 - network_mode: "wbms_network_mode" type indicating the desired Network mode for the connected BMU. By default, this parameter is disabled.
 - Static_ip_address: String indicating the Static IP address that shall be applied to the BMU. By default, this parameter is 0.0.0.0.

- Static_ip_netmask: String indicating the Static IP netmask that shall be applied to the BMU. By default, this parameter is 0.0.0.0.
 - Static_ip_gateway: String indicating the Static IP gateway that shall be applied to the BMU. By default, this parameter is 0.0.0.0.
 - Static_ip_dns_1: String indicating the Static IP DNS1 that shall be applied to the BMU. By default, this parameter is 0.0.0.0.
 - Static_ip_dns_2: String indicating the Static IP DNS 2 that shall be applied to the BMU. By default, this parameter is 0.0.0.0.
 - Import_dates: "0" indicates that the manufacturing and commissioning dates shall not be applied. "1" indicates that the dates shall be applied.
 - Manufacturing and Commissioning dates: Strings indicating the dates using a DD/MM/YY format.
- Return
 - result: Result of the apply setup process. The result is returned as "wbms_result" enumeration.
 - OK: It means that the WBMS-Toolkit has applied correctly the BMU Setup parameters to the BMU.
 - ERROR: It means that the WBMS-Toolkit has not been able to apply correctly the BMU Setup parameters to the BMU. In this situation, the cause of the problem can be any of the following points:
 - Incorrect BMU Setup parameters.
 - The WBMS-BMU is not connected to WBMS-Toolkit.
 - The connection settings from the WBMS-Toolkit are not the same as the physical connection with the BMU.
 - TIMEOUT: It means that the apply process has taken longer than expected. A typical situation may be that the BMU Setup parameters used are incorrect.
 - GETTING: It means that the WBMS-Toolkit is getting the current configured parameters from the BMU. For this reason, the new requested BMU setup configuration cannot be applied.

- WAITING: It means that the wBMS-Toolkit has not any BMU connected or is not authorized establish communication with it.

3.4. **wbms_set_connection_usb**

- Parameters
 - None
- Return
 - result: Result of the connection USB process. The result is returned as "wbms_result" enumeration.
 - OK: It means that the wBMS-Toolkit is connected correctly to the BMU.
 - ERROR: It means that the wBMS-Toolkit is not connected correctly to the BMU. In this situation, the cause of the problem can be any of the following points:
 - The wBMS-BMU is not connected to wBMS-Toolkit.
 - The connection settings from the wBMS-Toolkit are not the same as the physical connection with the BMU

3.5. **wbms_set_connection_can**

- Parameters
 - usb_to_can_converter: "wbms_can_converter" type indicating the used USB-to-CAN interface to connect the wBMS-Toolkit to the BMU.
 - canbus_baudrate: "wbms_can_baudrate" type indicating the used CAN bus baud rate to connect the wBMS-Toolkit to the BMU.
 - canbus_id: Integer type indicating the CAN node ID of the BMU that the wBMS-Toolkit shall be connected to.
 - can_toolkit_refresh: "wbms_can_toolkit_refresh" type indicating the period of the communication between the wBMS-Toolkit and the BMU. The FAST refresh sets the period to 1 second and the SLOW refresh sets the period to 5 seconds.

- Return
 - result: Result of the connection CAN process. The result is returned as “wbms_result” enumeration.
 - OK: It means that the WBMS-Toolkit is connected correctly to the BMU.
 - ERROR: It means that the WBMS-Toolkit is not connected correctly to the BMU. In this situation, the cause of the problem can be any of the following points:
 - The WBMS-BMU is not connected to WBMS-Toolkit.
 - The connection settings from the WBMS-Toolkit are not the same as the physical connection with the BMU

3.6.wbms_set_connection_bluetooth

- Parameters
 - device_id: Integer type indicating the device ID of the BMU that the WBMS-Toolkit shall be connected to.
- Return
 - result: Result of the connection Bluetooth process. The result is returned as “wbms_result” enumeration.
 - OK: It means that the WBMS-Toolkit is connected correctly to the BMU.
 - ERROR: It means that the WBMS-Toolkit is not connected correctly to the BMU. In this situation, the cause of the problem can be any of the following points:
 - The WBMS-BMU is not connected to WBMS-Toolkit.
 - The connection settings from the WBMS-Toolkit are not the same as the physical connection with the BMU.

3.7.wbms_start_logging

- Parameters
 - csv_file_path: String containing the path to the CSV file (.csv) where the WBMS-Toolkit shall be saving the information of the connected BMU.
- Return
 - result: Result of the start logging process. The result is returned as "wbms_result" enumeration.
 - OK: It means that the WBMS-Toolkit has started the logging correctly.
 - ERROR: It means that the WBMS-Toolkit has not started the logging process.

3.8.wbms_stop_logging

- Parameters
 - None
- Return
 - result: Result of the stop logging process. The result is returned as "wbms_result" enumeration.
 - OK: It means that the WBMS-Toolkit has stopped the logging correctly.
 - ERROR: It means that the WBMS-Toolkit has not stopped the logging process.

3.9.wbms_force_ocv_soc

- Parameters
 - None
- Return
 - result: The return value is always OK from the "wbms_result" enumeration.

3.10. **wbms_sign_in**

- Parameters
 - user: String containing the user credential to sign in.
 - password: String containing the password credential to sing in.
- Return
 - result: Result of the sign in process. The result is returned as “wbms_result” enumeration.
 - OK: It means that the WBMS-Toolkit is logged in correctly.
 - ERROR: It means that the WBMS-Toolkit is not logged in correctly. In this situation, the cause of the problem can be any of the following points:
 - The user credentials are incorrect.
 - The WBMS-Toolkit doesn't have Internet connection.

3.11. **wbms_set_connection_tcp**

- Parameters
 - tcp_ip: String containing the IP of the desired BMU. (i.e. 192.168.1.100)
- Return
 - result: Result of the connection TCP process. The result is returned as “wbms_result” enumeration.

3.12. **wbms_set_connection_mqtt**

- Parameters
 - serial_number: String containing the Serial Number of the desired BMU (i.e. 000451001000). Note that the SN length must be 12 digits, so 0's shall be placed before to reach the required length.
 - mqtt_endpoint_url: String containing the URL of the MQTT endpoint.
 - path_to_mqtt_certificate_path: String containing the path to the MQTT certificate (.pfx file).
 - authentication_password: String containing the password of the MQTT certificate (.pfx file).

- Return
 - result: Result of the connection MQTT process. The result is returned as “wbms_result” enumeration.

3.13. **wbms_toolki_api_poll**

- Parameters
 - Timeout: Maximum time that the function is waiting for the wBMS-Toolkit to open the wBMS-Toolkit-API before reporting error. The default time is 10 seconds.
- Return
 - result: Boolean variable that returns True when the wBMS-Toolkit-API is ready to be used before the timeout defined. Otherwise, it returns False.

3.14. **wbms_get_config_crc**

- Parameters
 - None
- Return
 - result: This value can be different depending on the scenario:
 - wbms_result.WAITING: Means that the wBMS-Toolkit has not received the Configuration CRC yet.
 - wbms_result.TIMEOUT: Means that an internal error has occurred during the process.
 - Configuration CRC: The values in decimal format.

3.15. **wbms_get_ip**

- Parameters
 - None
- Return
 - result: This value can be different depending on the scenario:
 - wbms_result.WAITING: It means that the wBMS-Toolkit has not received the assigned IP yet.

- `wbms_result.OK`: It means that the wBMS-Toolkit has received that the "Network mode" is "Disable" and for this reason no IP is assigned to the BMU.
- `"0.0.0.0"`: It means that the wBMS-Toolkit has received that the "Network mode" is different from "Disable" but there is not Ethernet connection and for this reason no IP is assigned to the BMU.
- `Correct IP`: It means that the wBMS-Toolkit has received that the "Network mode" is different from "Disable" and there is Ethernet connection.

4. Enumerations

The defined enumerations become available after importing, as explained in section 2, the "wbms_api.wbms_toolkit_types".

4.1. wbms_result

- OK = 0
- ERROR = 1
- TIMEOUT = 2
- GETTING = 3
- WAITING = 4

4.2. wbms_connection_types

- USB = 0
- MQTT = 1
- TCP = 2
- BLE = 4 (Bluetooth)
- CAN = 5

4.3. wbms_can_baudrate

- CAN_50kbps = 1
- CAN_100kbps = 2
- CAN_125kbps = 3
- CAN_250kbps = 4
- CAN_500kbps = 5
- CAN_1000kbps = 6

4.4. wbms_can_converter

- KVASER = 0
- PEAK = 1

4.5. wbms_can_toolkit_refresh

- FAST = 0
- SLOW = 1

4.6. wbms_ble_mode

- ENABLED = 0
- DISABLED = 1

4.7. wbms_logging

- STOP = 0
- START = 1

4.8. wbms_network_mode

- DISABLE = 0
- STATIC IP = 1
- DHCP = 2

5. Examples

Some examples are available in the Wattius Services repository. The following subchapters provide a brief description of their content.

5.1. `exampleScript_apply_setup.py`

This example configures the BMU Setup parameters to the connected BMU. The user shall introduce all the options in the console :

- CAN bus: baud rate and can node ID.
- Bluetooth mode
- Network: mode and Static IP settings.
- Dates: Manufacturing and Commissioning dates.

5.2. `exampleScript_ble_connection.py`

This example connects the WBMS-Toolkit to a BMU via Bluetooth. The user shall only introduce the device ID of the desired BMU.

5.3. `exampleScript_can_connection.py`

This example connects the WBMS-Toolkit to a BMU via CAN. The user shall introduce the CAN bus baud rate, the USB-to-CAN interface type and the CAN node ID of the desired BMU.

5.4. `exampleScript_start_logging.py`

This example sends a command to the WBMS-Toolkit to start saving the information of the connected BMU to the desired CSV file, whose path shall be introduced on the console. If the CSV file is not created, the WBMS-Toolkit will create it automatically.

5.5. `exampleScript_stop_logging.py`

This example sends a command to the WBMS-Toolkit to stop the logging.

5.6.exampleScript_update_firmware.py

This example updates a desired firmware to the connected BMU. The user shall introduce the path to the desired firmware file on the console

5.7.exampleScript_upload_configuration.py

This example uploads a desired BMU Settings to the connected BMU. The user shall introduce the path to the desired BMU Settings file on the console.

5.8.exampleScript_usb_connection.py

The example connects the WBMS-Toolkit to a BMU via USB.

5.9.exampleScript_force_ocv_soc.py

This example forces the BMU to calibrate the SoC to the current OCV SoC.

5.10. exampleScript_sign_in.py

This example allows to sign in to the WBMS-Toolkit remotely. The user and the password credentials shall be introduced.

5.11. exampleScript_tcp_connection.py

This example connects the WBMS-Toolkit to a BMU via TCP. The user shall introduce the IP of the desired BMU on the console.

5.12. exampleScript_mqtt_connection.py

This example connects the WBMS-Toolkit to a BMU via MQTT. The user shall introduce the Serial Number of the desired BMU, the URL of the MQTT endpoint, the path where the MQTT certificate (.pfx file) is placed and the MQTT certificate password.

Note that the SN length must be 12 digits, so 0's shall be placed before to reach the required length.

5.13. exampleScript_get_config_crc.py

This example requests the Configuration CRC until the returned value is different from wbms_result.WAITING. This example has a 30 seconds timeout after which an

exception is raised and the script is finished. If the Configuration CRC has been get correctly, it is printed on the console in decimal and hexadecimal formats.

Note: This example is constantly trying to get the configuration CRC. This strategy forces to print several messages on the console. To avoid this behaviour, a delay can be applied before every retry attempt.

5.14. **exampleScript_get_ip.py**

This example requests the assigned IP to the BMU until the `wbms_result` value returned value is not WAITING nor OK and the IP is different from "0.0.0.0" or after 30 seconds receiving, at least, one of the previous conditions. The IP is printed on the console.

Note: This example is constantly trying to get the assigned IP. This strategy forces to print several messages on the console. To avoid this behaviour, a delay can be applied before every retry attempt

5.15. **exampleScript_end_of_line.py**

This example represents a common End of Line process. It uses different functionalities of the WBMS-Toolkit-API to fulfill typical steps required on the End of Line. The steps performed are the following:

1. *Environment definition:* The first step of the example is requesting all the information needed during the execution of the process. This information is required through the console. Nevertheless, the example can be modified to use a static environment.
2. *wBMS-Toolkit initialization:* In this second step, the wBMS-Toolkit is automatically opened and logged in using the defined credentials. It remains in this step until the wBMS-Toolkit-API is ready to be used.

Note: The current implementation requires Internet connection every time that this step is executed. To avoid having Internet connection every time, open the wBMS-Toolkit manually and select the "Keep me logged in" checkbox during the log in. Then, every time that the wBMS-Toolkit is opened automatically, the user credentials aren't required.

3. *Update firmware(s):* In this step, the Safety firmware is updated. In the case of the wBMS-HX BMU, the Gateway firmware is updated as well.

Note: In this step, when the BMU is already in bootloader, the first connection can return error because of the wBMS-Toolkit not being able to connect to the BMU. This is expected behaviour as the BMU has no valid firmware uploaded.

4. *Upload configuration:* The configuration file and the BMU Setup are uploaded on this step. If all the process has finished successfully, the configuration CRC is printed on the console using decimal and hexadecimal formats.

Note: This step is constantly trying to apply the BMU setup and uploading the configuration. This strategy forces to print several messages on the console. To avoid this behaviour, a delay can be applied before every retry attempt.

5. *Upload MQTT connectivity settings:* This step is only executed in case of using the wBMS-HX BMU. The MQTT certificates and the MQTT broker URL are saved on the BMU. Furthermore, the MQTT connection settings are also uploaded. This step doesn't use the wBMS-Toolkit-API, but the script is connected directly to the Gateway MCU from the BMU to save the configuration parameters.
6. *Close wBMS-Toolkit:* Finally, the wBMS-Toolkit is automatically closed.

6. Compatibilites

wBMS-Toolkit-API versions	wBMS-Toolkit versions
V1.0.0	From V3.8.3 to Lastest Version